



École Polytechnique Fédérale de Lausanne

Trustless AI Agents
Can we achieve Verifiable Inference
using ZK?

by Youssef Jeddi

Master Project Report

Prof. Rachid Guerraoui
Project Advisor

Zicong Hong
Project Supervisor

EPFL IC DCL
INR
CH-1015 Lausanne

June 3, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Background on Verifiable Inference	4
1.3	Project Goals	4
2	Background	5
2.1	Zero-Knowledge Proofs	5
2.2	ZK Proofs for Machine Learning	6
2.3	EZKL	6
2.4	Related Systems	7
3	System Architecture	7
3.1	Overview	7
3.2	Model Preparation	8
3.3	Proof Generation Pipeline	8
3.4	On-Chain Verification	9
3.5	Implementation	10
4	Benchmarking Study	10
4.1	Methodology	10
4.2	Results	11
4.3	Findings	12
4.4	Plots	13
4.5	Discussion	14
5	Optimization Investigation	14
5.1	Research Question	15
5.2	Hypothesis 1: Scale Factor Reduction	15
5.3	Scale Factor Experiment	15
5.4	Hypothesis 2: Pre-Quantization	16
5.5	Pre-Quantization Experiment	16
5.6	Why Both Approaches Failed	17
6	Related Work & Comparison	17
6.1	zkLLM Overview	17
6.2	Key Innovations	18
6.3	Why Their Approach Works	18
6.4	Why We Could Not Apply These Techniques	19

6.5	Implications	19
7	Conclusion & Future Work	19
7.1	Summary of Contributions	20
7.2	Key Findings	20
7.3	Limitations	20
7.4	Future Work	21
	Bibliography	22

Introduction

1.1 Motivation

We increasingly delegate consequential decisions to AI. Models recommend medical treatments, screen loan applications, moderate online content, and, in decentralized finance, autonomously trade assets on our behalf. In almost every case the model runs on infrastructure we do not control, behind an API we cannot inspect. When a provider such as a large model vendor returns an answer, we are asked to take it on faith that the answer was produced by the model they claim to be running, on the input we actually sent, without tampering.

This is the *black box trust* problem. A malicious or careless provider could silently substitute a cheaper model, bias an outcome, censor a result, or, in a financial setting, act against the user's interest while presenting a plausible output. The user has no cryptographic means of detecting any of this. As AI agents are given more autonomy and, increasingly, control over real assets, the absence of verifiability becomes a serious liability.

1.2 Background on Verifiable Inference

Zero-Knowledge Machine Learning (zkML) offers exactly such a mechanism. The idea is to accompany each inference with a *zero-knowledge proof* that cryptographically attests that a specific model, with specific committed weights, was executed on a specific input to produce a specific output. The proof can be verified by anyone, including a smart contract on a public blockchain, without re-running the model and without trusting the party that generated it.

Conceptually this creates a *leash* on the agent. Before any interaction, the parties commit to a model by publishing a cryptographic fingerprint of its architecture and weights. Every subsequent inference must be accompanied by a proof that is only valid with respect to that committed model. If the agent runs anything else, the proof fails verification and the action is rejected. The trust assumption shifts from “believe the provider” to “believe the ZKP math,” which is a far stronger foundation.

1.3 Project Goals

This project investigates whether verifiable inference is practical today using accessible, off-the-shelf tooling, and where its limits lie. The work makes three contributions.

First, we build a complete end-to-end verifiable inference pipeline on top of the EZKL toolchain: it takes a trained neural network, uses EZKL to generate a zero-knowledge proof of an inference, and verifies that proof on the Ethereum Sepolia testnet through an automatically generated smart contract.

Second, we conduct a systematic benchmarking study of how proof generation cost scales with model size and architecture. Using the EZKL toolchain, we measure proving time, memory consumption, and circuit size across four families of models, and we identify the practical limits of the approach on commodity hardware.

Third, we investigate methods for reducing proving time. Guided by our benchmarking results, we test whether value-level techniques, namely reducing the fixed-point scale factor and pre-quantizing model weights, can lower proving cost. We report negative results for both, analyze why they fail, and contrast our findings with the state of the art zkLLM system to explain what a successful optimization actually requires.

Background

This chapter introduces the concepts and tools needed to understand the rest of the report: zero-knowledge proofs, the specific challenges of applying them to machine learning, the EZKL toolchain we build on, and the broader landscape of related systems.

2.1 Zero-Knowledge Proofs

A zero-knowledge proof is a protocol between a *prover* and a *verifier* that allows the prover to convince the verifier that a statement is true without revealing anything beyond the truth of the statement itself. Three properties define such a system. *Completeness* means that an honest prover can always convince the verifier of a true statement. *Soundness* means that a dishonest prover cannot convince the verifier of a false statement, except with negligible probability. *Zero-knowledge* means that the proof reveals nothing about the prover’s private inputs beyond the validity of the statement.

Modern succinct proof systems, such as zk-SNARKs [2], additionally guarantee that proofs are small and fast to verify, even when the underlying computation is large. This asymmetry is the key practical property: proof generation may be expensive, but verification is cheap and constant or near-constant in cost. It is precisely this asymmetry that makes on-chain verification feasible, since

a smart contract can check a proof without re-executing the computation that produced it.

To be provable, a computation must first be expressed as an *arithmetic circuit*: a system of polynomial constraints over a finite field. The prover demonstrates that it knows an assignment of values, the *witness*, that satisfies every constraint. Any computation we wish to prove must therefore be translated into this constraint form.

2.2 ZK Proofs for Machine Learning

Applying zero-knowledge proofs to neural network inference introduces two structural difficulties.

The first is numerical representation. Neural networks operate on floating-point numbers, but arithmetic circuits operate on integers in a finite field. Bridging this gap requires *quantization*: floating-point values are scaled by a fixed factor and rounded to integers, a representation known as fixed-point arithmetic. The choice of scale factor trades precision against circuit complexity, a trade-off we examine directly in Chapter 5.

The second difficulty is non-arithmetic operations. Neural networks rely heavily on operations that are not naturally polynomial, such as ReLU, GELU, and Softmax. These cannot be expressed directly as arithmetic constraints. The standard technique to handle them is the *lookup argument*, in which a precomputed table of valid input-output pairs is committed, and the prover shows that each computed pair appears in the table. Lookup arguments make non-arithmetic operations provable, but they add overhead, and for operations with very large input domains the tables become a serious bottleneck.

These two difficulties, quantization and non-arithmetic operations, are the central technical obstacles in zkML, and they reappear throughout this report.

2.3 EZKL

EZKL [10] is an open-source toolchain that compiles trained machine learning models into zero-knowledge circuits and generates proofs of correct inference. It accepts models in the ONNX format, exported from frameworks such as PyTorch, and is built on the Halo2 proof system [9].

The appeal of EZKL for this project is its accessibility. It abstracts away the underlying cryptography, so an engineer can compile a model, generate a proof, and obtain a deployable verifier with a small number of high-level commands. Critically, EZKL automatically generates a Solidity verifier contract, which can be deployed to any EVM-compatible blockchain. This makes it the only tool we are aware of that takes an existing model file and produces, end to end, a deployable on-chain

verifier, which is exactly what a trustless agent requires.

EZKL handles quantization through a calibration step that selects a fixed-point scale factor, and it handles non-arithmetic operations through Halo2’s lookup arguments. Its generality is also its limitation: because it is not specialized to any particular model architecture, it cannot exploit the structural shortcuts that custom proof systems use. This tension between accessibility and specialization is a recurring theme of our results.

2.4 Related Systems

EZKL sits within a broader ecosystem of zkML systems. Early work such as zkCNN [4] demonstrated specialized proof protocols for convolutional networks, achieving efficient proofs by exploiting the structure of convolutions. zkML [3] extended verifiable inference toward larger models and was among the first to handle networks at the scale of GPT-2. The current state of the art for large models is zkLLM [8], which introduces custom cryptographic protocols specifically designed for transformer inference and scales to models with thirteen billion parameters; we examine zkLLM in detail in Chapter 6, since it provides the natural point of comparison for our optimization findings.

Finally, it is worth situating verifiable inference within emerging standards for agent accountability. Proposals such as ERC-8004 [1] explore on-chain identity and trust mechanisms for autonomous agents. Verifiable inference is complementary to such efforts: where identity standards establish *who* an agent is, zero-knowledge inference proofs establish *that* an agent computed honestly, and the two together move toward genuinely trustworthy autonomous systems.

System Architecture

This chapter describes the end-to-end system we built to take a trained neural network from a PyTorch definition all the way to an on-chain verifiable inference. We first give an overview of the full pipeline, then describe each stage in turn, and conclude with notes on the implementation.

3.1 Overview

The system is organized into three phases, illustrated in Figure 3.1. In the *model preparation* phase, a PyTorch model is exported to the ONNX [5] format that EZKL consumes. In the *proof generation*

phase, EZKL compiles the model into a zero-knowledge circuit and produces a proof that a specific input was run through that circuit to produce a specific output. In the *verification* phase, the proof is checked, either locally or, more importantly, on-chain by a smart contract deployed to the Sepolia testnet.

The defining property of the pipeline is the asymmetry between proving and verification. The heavy computation happens once, off-chain, during proof generation. Verification is lightweight and can therefore run inside the constrained environment of a blockchain. This is what makes trustless, publicly checkable inference practical.

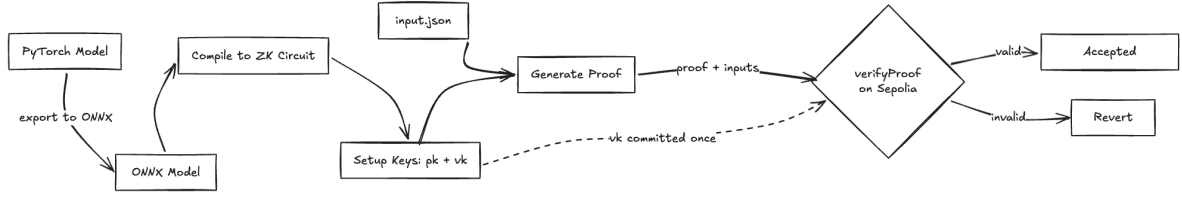


Figure 3.1: End-to-end pipeline of the system

3.2 Model Preparation

The pipeline begins with a model defined and trained in PyTorch [6]. EZKL does not consume PyTorch models directly; it requires the ONNX interchange format. The model is therefore exported using `torch.onnx.export`.

Two practical constraints shaped this stage. First, EZKL’s ONNX frontend reliably supports a specific operator set, so we export with `opset_version=11` and force the legacy exporter with `dynamo=False` to avoid incompatibilities. Second, and more significantly, we found empirically that EZKL only accepts FP32 models: attempting to export in FP16 or BF16 causes the toolchain to fail at graph translation, before any circuit is generated. This finding, discussed further in Chapter 5, means the input model is always FP32, and any reduced precision must be applied through EZKL’s own quantization rather than through the export format.

3.3 Proof Generation Pipeline

Once an ONNX model is available, EZKL drives a sequence of stages that together produce a proof. Each stage has a distinct role.

Settings generation analyzes the ONNX graph and determines the cryptographic parameters

needed to represent it as a circuit. *Calibration* runs the model on sample inputs to select the fixed-point quantization parameters, choosing scale factors that convert floating-point operations into integer arithmetic while minimizing numerical error; it is at this stage that EZKL reports its numerical fidelity. *Compilation* turns the model and settings into a concrete arithmetic circuit. The *structured reference string* (SRS), the public parameters underpinning the proof system, is then loaded. *Setup* derives the proving key and the verification key from the compiled circuit and the SRS; the verification key is the cryptographic fingerprint of the model.

With keys in hand, *witness generation* runs the model on a specific input and records every intermediate value of the computation. *Proving* then consumes the witness to construct the zero-knowledge proof; this is where the heavy cryptography occurs and, as our benchmarks show, where the majority of time and memory is spent. Finally, *verification* checks the proof against the verification key, which is fast in comparison to proving.

3.4 On-Chain Verification

The component that makes the system trustless rather than just verifiable is on-chain verification. EZKL generates a Solidity verifier contract directly from the verification key. We compile this contract and deploy it to the Sepolia testnet.

The verification key encodes the committed model. Because it is baked into the deployed contract, the model cannot be changed without redeploying a new contract. Each inference proof is then submitted to the contract through its `verifyProof` function, together with the public inputs, which encode the input and output values of the inference. The contract returns true only if the proof is valid with respect to the committed model.

This arrangement provides three concrete guarantees, which together constitute the trust model of the system. It guarantees the *right model*, because the proof is only valid against the verification key committed in the contract. It guarantees the *right input*, because the input is fixed in the public instances of the proof and cannot be altered afterward. And it guarantees the *right computation*, because the proof verifies only if the entire arithmetic circuit is satisfied. A user who knows the input they submitted can confirm that the public instances match it.

A single deployed contract verifies arbitrarily many proofs for its committed model. Each new inference produces a fresh proof with fresh public instances, all checked by the same contract.

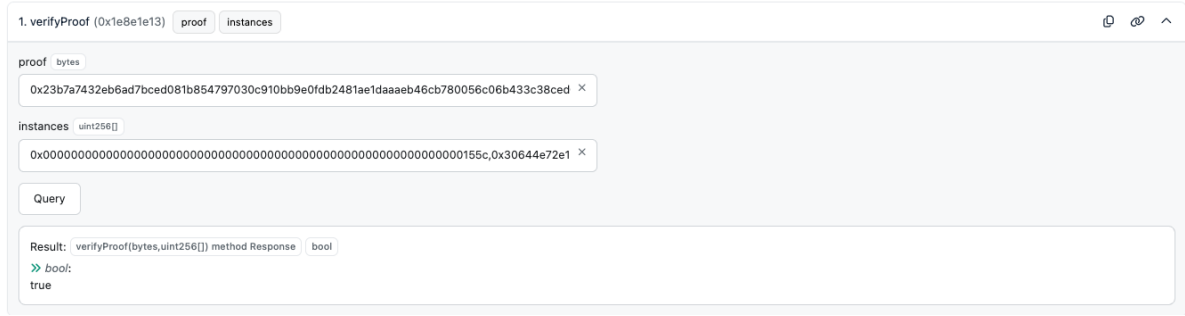


Figure 3.2: On-chain verification on etherscan.

3.5 Implementation

The system is implemented in Python, orchestrating EZKL through its Python bindings, with PyTorch for model definition and export. The full source code is available in the project repository.¹

Benchmarking Study

The major contribution of this project is at benchmarking study of how zero-knowledge proof generation scales with model size and architecture in EZKL.

4.1 Methodology

To isolate the variables that drive proving cost, we constructed twenty-four PyTorch models spanning four architecture families, plus a baseline model.

The *MLP width series* (mlp_w01 through mlp_w08) fixes the depth at three hidden layers and grows the hidden width from 16 up to 2048, isolating the effect of width. The *MLP depth series* (mlp_d01 through mlp_d07) fixes the width at 128 and grows the depth from 2 up to 20 hidden layers, isolating the effect of depth. The *CNN series* (cnn01 through cnn06) ranges from a toy single-convolution model up to a mini-VGG and a six-convolution batch-normalized network. The *attention series* (attn01, attn02) implements manual query/key/value projections with Softmax, for one and two self-attention layers respectively.

¹<https://github.com/youssef-jeddi/research-project>

Because proof generation time depends on the structure of the computation rather than on the specific numerical values of the weights, the models are instantiated with random weights; no training is required for the benchmarking purpose. Each model is driven through the complete EZKL cycle by an automated harness, `run_benchmark.py`, which performs export, settings generation, calibration, compilation, SRS loading, setup, witness generation, proving, and verification. The harness records per-model metrics including parameter count, the circuit size parameter `logrows`, artifact sizes, peak resident memory, and the timing of each pipeline stage, saving results incrementally to CSV. A companion script, `analyze_results.py`, produces the plots and summary tables.

All experiments were run on a MacBook with 18 GB of RAM. This consumer-hardware setting is deliberate: one goal of the study is to establish what is achievable without specialized infrastructure.

4.2 Results

Table 4.1 reports the results for the eighteen models that completed successfully. All produced valid, verified proofs. The table shows parameter count, proving time, peak memory, and the circuit size parameter `logrows` for each model.

Model	Architecture	Params	Prove (s)	Peak RSS	logrows
model1	mlp_baseline	234	0.77	676 MB	15
mlp_w01	mlp_width	884	0.77	676 MB	15
mlp_w02	mlp_width	2,788	0.79	744 MB	15
mlp_w03	mlp_width	9,668	0.85	763 MB	15
mlp_w04	mlp_width	35,716	0.97	780 MB	15
mlp_w05	mlp_width	136,964	3.24	1.97 GB	17
mlp_w06	mlp_width	536,068	12.24	5.64 GB	19
mlp_w07	mlp_width	2,120,708	58.44	8.56 GB	21
mlp_d01	mlp_depth	19,204	0.89	~1 GB	15
mlp_d02	mlp_depth	52,228	1.76	~1 GB	16
mlp_d03	mlp_depth	85,252	3.15	~1 GB	16
mlp_d04	mlp_depth	118,276	3.47	~1 GB	17
mlp_d05	mlp_depth	184,324	6.52	~1 GB	17
mlp_d06	mlp_depth	250,372	6.71	~1 GB	17
mlp_d07	mlp_depth	316,420	6.96	~1 GB	18
attn01	attention	4,356	5.39	~1 GB	16
attn02	attention	33,796	27.02	~2 GB	18
cnn01	cnn	1,652	~6.5	~1 GB	17

Table 4.1: Benchmark results across eighteen models and four architecture families. All proofs were generated and verified successfully.

The width series shows a clear progression: from sub-second proving at a few thousand parameters up to nearly a minute of proving and 8.56 GB of memory at two million parameters. The depth

series shows a similar climb at comparable parameter counts. The attention and CNN models stand out: `attn02`, with only 34K parameters, takes 27 seconds to prove, comparable to a 300K-parameter MLP, and the CNN models prove disproportionately slowly relative to their parameter counts.

4.3 Findings

Six findings emerge from the data.

First, MLP proving cost scales cleanly with parameter count. On a log-log plot the relationship is close to a slope of one, meaning that doubling the number of parameters roughly doubles the proving time.

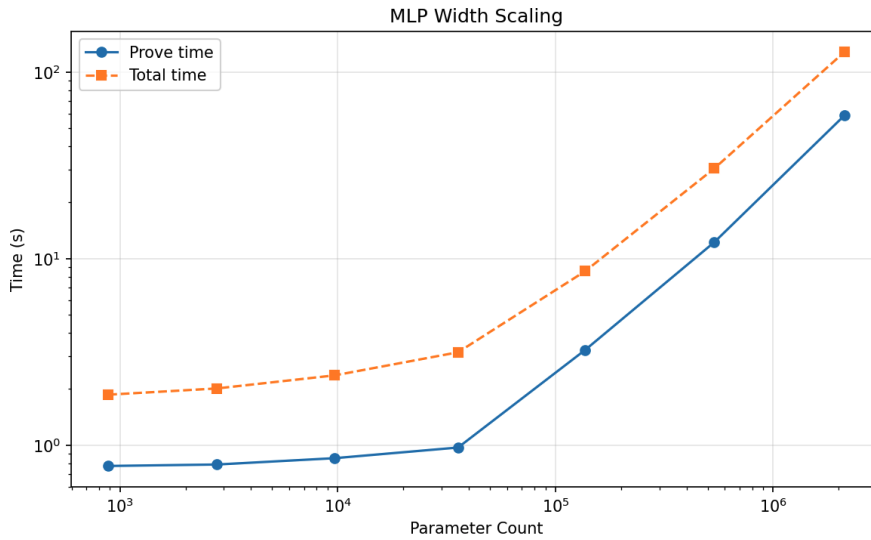


Figure 4.1: Proving time and total pipeline time versus parameter count for the MLP width series.

Second, width and depth behave similarly for MLPs at comparable parameter counts. There is no surprising architectural penalty for one over the other; what matters is the total parameter count, not how those parameters are distributed.

Third, setup and proving dominate the pipeline. Together they account for more than ninety percent of wall-clock time on large models. Export, settings generation, witness generation, and verification are all comparatively cheap and roughly constant in cost.

Fourth, attention is substantially more expensive than dense layers, roughly five times more expensive per parameter than an MLP. This is likely attributable to the Softmax operation, a non-linear function that does not map cleanly to polynomial constraints and therefore inflates the circuit.

Fifth, convolutional networks explode circuit size far more than their parameter counts would suggest. A small CNN with only eleven thousand parameters operating on 28×28 inputs triggered an out-of-memory failure during setup. The circuit must unroll every convolution position, so input resolution matters as much as parameter count.

Sixth, memory rather than compute is the binding constraint. Peak memory tracks the circuit size parameter $\log_{\text{rows}}$ rather than proving time, and since memory scales with $2^{\log_{\text{rows}}}$, a fourfold increase in parameters can push $\log_{\text{rows}}$ up by one or two and thereby double or quadruple memory usage. This also explains why GPUs do not help: Halo2’s prover is CPU-bound, so additional RAM, not additional parallel compute, is what extends the reachable model size.

4.4 Plots

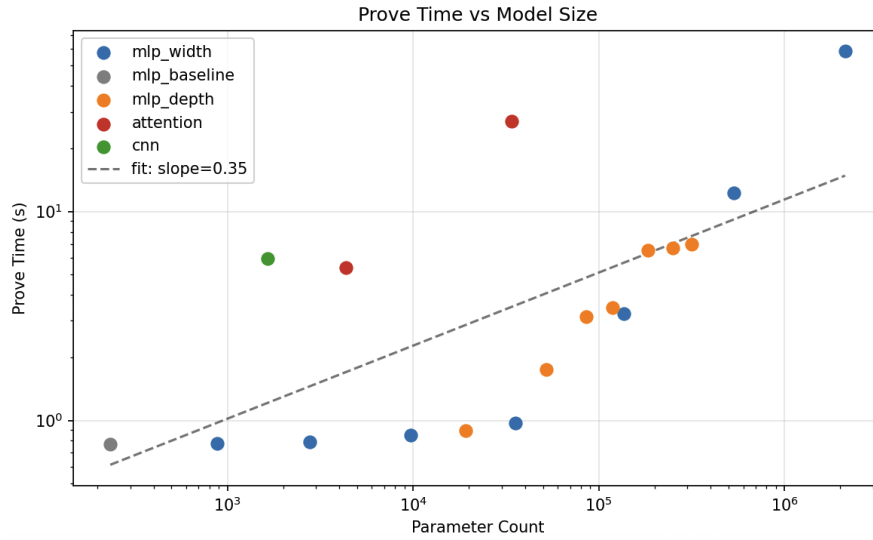


Figure 4.2: Proving time versus parameter count across architecture families.

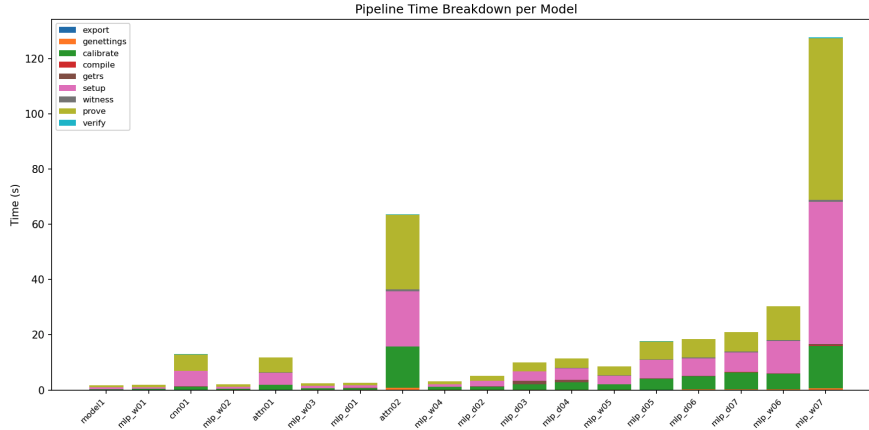


Figure 4.3: Pipeline time per model.

4.5 Discussion

The benchmarks establish both that accessible zkML works and where it stops working. On commodity hardware, MLPs up to roughly two million parameters are provable, while convolutional and attention-heavy models hit the memory wall far sooner relative to their size. The bottleneck is consistently memory driven by circuit size, not raw compute.

These results raise the natural question that motivates the next chapter. If proving cost is dominated by circuit size, and circuit size is in turn driven by the structure and precision of the computation, can we reduce proving time by reducing that precision? Two levers present themselves immediately within EZKL’s interface: the fixed-point scale factor used during quantization, and pre-quantization of the model’s weights. The following chapter tests both.

Optimization Investigation

The second major contribution of this project is an investigation into reducing proving time. The benchmarking study established that proving cost is driven by circuit size. The natural follow-up question is whether we can shrink that cost using the levers EZKL exposes, without modifying the proof system itself.

5.1 Research Question

Given the scaling behavior observed in Chapter 4, can we reduce proving time for a fixed model without changing the underlying proof system? We restrict attention to value-level techniques accessible through EZKL’s standard interface, since these are what a practitioner could realistically apply. Two such techniques present themselves: reducing the fixed-point scale factor used during quantization, and pre-quantizing the model’s weights to a coarse set of values before export.

For all experiments in this chapter we use `mlp_w05`, a three-hidden-layer MLP of the form $16 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 4$ with roughly 137K parameters. It is large enough that its proving time (around three seconds) is meaningfully above noise, yet small enough to iterate on quickly. We measure both proving time and accuracy, the latter by comparing the quantized circuit’s outputs against the original model’s outputs on a fixed set of one hundred test inputs.

5.2 Hypothesis 1: Scale Factor Reduction

EZKL converts floating-point values to fixed-point integers by multiplying by a scale factor of 2^s and rounding, with $s = 13$ by default in our calibrations. Higher scale preserves more precision but produces larger integers in the circuit; larger integers require more constraints to handle without overflow, which should in principle increase circuit size and therefore proving time. The hypothesis is that the default scale is over-conservative for simple models, and that lowering it would reduce proving time at an acceptable cost in accuracy.

5.3 Scale Factor Experiment

We swept the scale factor across the values $\{4, 6, 8, 10, 13, 16\}$, setting both the input scale and the parameter scale to each value in turn, and ran the full pipeline for each. For every setting we recorded the circuit size parameter `logrows`, the proving time, the proof size, and three accuracy measures: mean absolute error and maximum absolute error against the FP32 reference, and the prediction agreement, defined as the fraction of the hundred test inputs for which the quantized circuit and the FP32 model selected the same output class.

The results, summarized in Table 5.1, refute the hypothesis. Proving time is flat across the entire range. Crucially, `logrows` remains constant at 17 for every scale, the circuit size never changes, and the compiled circuit and proving key are byte-for-byte identical across settings. Meanwhile accuracy degrades steeply at low scales: prediction agreement falls to 91 percent at scale 8 and to 62 percent at scale 4. Lowering the scale therefore buys nothing in speed while sacrificing accuracy.

Metric	s=4	s=6	s=8	s=10	s=13	s=16
logrows	17	17	17	17	17	17
prove (s)	3.46	3.43	3.34	3.43	3.64	3.57
mean abs err	0.0722	0.0083	0.0024	4.6e-4	7.8e-5	1.2e-5
pred. agree	0.62	0.83	0.91	0.99	0.99	1.00

Table 5.1: Scale factor sweep on `mlp_w05`.

5.4 Hypothesis 2: Pre-Quantization

The second hypothesis attacks the problem from a different angle. Rather than asking EZKL to use a coarser scale, we coarsen the model itself before EZKL sees it. If the weights are constrained to a small set of discrete values, perhaps EZKL can allocate a smaller or simpler circuit because there is less numerical information to represent. Because EZKL only accepts FP32 models, we cannot export an integer-quantized ONNX file directly; instead we quantize the weights to a coarse grid and cast them back to FP32, so the ONNX graph remains FP32 while the actual weight values carry the precision loss.

5.5 Pre-Quantization Experiment

We quantized the weights of `mlp_w05` to progressively coarser grids: from full FP32 (no quantization) down through 8-bit, 6-bit, 4-bit, 3-bit, and 2-bit levels, to a binary representation with only two distinct values per layer. For each level we ran the full pipeline at the default scale of 13 and recorded circuit size, proving time, and accuracy.

Metric	none	int8	int6	int4	int3	int2	binary
unique weights	135668	1010	249	57	25	9	8
logrows	17	17	17	17	17	17	17
prove (s)	3.45	3.33	3.45	3.35	3.38	3.33	3.45
pred. agree	0.99	1.00	0.99	0.92	0.78	0.68	0.32

Table 5.2: Pre-quantization sweep on `mlp_w05`.

The results, summarized in Table 5.2, again refute the hypothesis, and even more starkly. Reducing the model from over 135,000 unique weight values down to just eight had no effect whatsoever on circuit size or proving time: `logrows` stayed at 17 and proving time varied only within noise. Accuracy held up well down to the 6-bit level and then collapsed, but this degradation comes entirely from the quantization of the model itself, not from EZKL, and it buys no speedup in return.

A useful secondary observation is that EZKL’s fixed-point conversion is faithful: the prediction agreement of the EZKL-processed model tracks that of the quantized PyTorch model almost exactly,

confirming that EZKL adds negligible error of its own on top of whatever quantization was applied.

5.6 Why Both Approaches Failed

Both experiments point to the same underlying cause. EZKL’s proving cost is determined by the *structural* properties of the computational graph, the number and type of operations and the dimensions of each layer, and not by the numerical precision or value distribution of the weights. The circuit allocates a fixed number of constraints to each operation, and those slots cost the same to prove whether they are filled by a high-precision float or a coarse integer. Halo2, the proof system underlying EZKL, has no mechanism to exploit value-level redundancy or sparsity: a weight of 0.347281 and a weight of 1 are equally expensive.

This is not a tuning failure but an architectural property of the proof system. It also explains the constancy of logrows across every configuration in both experiments. Since circuit size is fixed by structure, and proving cost is fixed by circuit size, no amount of value-level coarsening can move the needle. To actually reduce proving time, one must change the structure, by using a smaller architecture, or change the proof system itself.

Related Work & Comparison

Our optimization investigation produced two negative results: neither scale factor reduction nor pre-quantization reduces proving time in EZKL, because proving cost is fixed by circuit structure rather than value precision. This chapter turns those negative results into a substantive contribution by comparing them against zkLLM. The comparison clarifies precisely what a successful optimization requires, and why it lies beyond the reach of the tools we used.

6.1 zkLLM Overview

zkLLM is a zero-knowledge proof system designed specifically for large language models. Where general-purpose frameworks struggle beyond a billion parameters, zkLLM generates a correctness proof for the entire inference process of a 13 billion parameter model in under 15 minutes, producing a proof under 200 KB that verifies in one to three seconds. These numbers are far beyond anything achievable with EZKL on the kind of models we benchmarked, and the gap is instructive.

6.2 Key Innovations

zkLLM’s performance rests on four innovations, each of which addresses a cost that our own benchmarks identified.

The first is **tlookup**, a parallelized lookup argument for non-arithmetic operations. The standard lookup arguments used by Halo2, and therefore by EZKL, are based on univariate polynomial constructions that are inherently sequential. tlookup instead exploits a rational-function identity for set inclusion that can be verified at a single random point and that parallelizes across the entire tensor. Handling activation functions such as ReLU and GELU therefore stops being a sequential bottleneck.

The second is **zkAttn**, a proof protocol crafted specifically for the attention mechanism. Our benchmarks found attention to be roughly five times more expensive per parameter than dense layers, owing to Softmax. zkAttn tackles Softmax directly by decomposing its exponentiation using the identity $\exp(a + b) = \exp(a) \exp(b)$: each input is split into segments, the exponential of each small segment is obtained from a small lookup table, and the results are multiplied. This turns an intractable monolithic lookup into several feasible small ones. Normalization, which would otherwise require highly non-arithmetic operations, is verified indirectly by checking that the output rows sum to the expected value within a tolerance.

The third is the use of **sumcheck protocols over multilinear extensions** in place of Halo2’s univariate polynomial approach. Sumcheck-based proofs are naturally parallelizable and align well with the tensor structure of neural network computations; dedicated sumchecks for matrix multiplication can even run faster than the multiplication itself.

The fourth is a **CUDA implementation**. Because the redesigned protocols are parallel by construction, they map onto GPU hardware, which the sequential structure of Halo2 cannot exploit. This is the engineering layer that turns the algorithmic improvements into wall-clock speedups.

6.3 Why Their Approach Works

The crucial observation is that zkLLM and our experiments diagnose the same problem but act at different layers. We found that proving cost is fixed by circuit structure and that value-level tuning cannot change it. zkLLM accepts this and responds by *redesigning the circuit construction itself*: tlookup and zkAttn change how operations are encoded into constraints, and the sumcheck and CUDA choices change how those constraints are proven. They do not tune parameters within an existing proof system; they replace the proof system with one tailored to the structure of neural networks. This is exactly the kind of structural change that our analysis concluded was necessary.

6.4 Why We Could Not Apply These Techniques

None of zkLLM’s innovations are accessible through EZKL. They live below the level of EZKL’s API, inside the proof system itself. Adopting tlookup or zkAttn would mean replacing Halo2’s lookup arguments and circuit construction, and adopting the sumcheck and CUDA design would mean replacing the proving backend wholesale. Reproducing any of this would amount to reimplementing zkLLM, a research engineering effort with cryptographic expertise, and is out of scope for a semester project. Moreover, zkLLM’s own implementation [7] is a research prototype: it is hardcoded for specific model families, requires high-end GPUs, is no longer maintained, and produces proofs in a format incompatible with the Solidity verifier path that our on-chain system depends on. It is therefore valuable as a point of comparison and as a source of insight, but not as a tool we could have plugged into our pipeline.

6.5 Implications

The comparison yields a clear and, we believe, useful conclusion. Value-level optimizations within general-purpose zero-knowledge frameworks have a fundamental ceiling. Because proving cost is fixed by circuit structure, and general-purpose frameworks construct that structure without awareness of the model’s semantics, there is no value-level knob that reduces it. Meaningful reductions in proving time require either reducing the structural complexity of the model, through smaller architectures or pruning, or moving to a proof system specialized for LLMs, as zkLLM demonstrates. For practitioners, the practical takeaway is concrete: do not invest effort in tuning scale factors or pre-quantizing weights to speed up EZKL, because neither will help; invest instead in model size reduction or in AI-specialized proof systems. We expect such specialized systems to continue outperforming general-purpose frameworks for verifiable inference for the foreseeable future.

Conclusion & Future Work

This project set out to determine whether trustless, verifiable AI inference is practical with accessible tooling, and where its limits lie. We answered both halves of that question: we built a working system that demonstrates the concept end to end, and we mapped the boundaries of the approach through systematic benchmarking and a focused optimization study.

7.1 Summary of Contributions

The project made three contributions. We built a complete end-to-end zkML system that takes a PyTorch model, generates a zero-knowledge proof of an inference with EZKL, and verifies that proof on-chain through a Solidity contract deployed to the Sepolia testnet. We conducted a systematic benchmarking study across four architecture families and eighteen successfully proven models, characterizing how proving time, memory, and circuit size scale and identifying the practical limits on commodity hardware. And we carried out an optimization investigation that tested two value-level techniques for reducing proving time, reported negative results for both, and explained the negative results through comparison with zkLLM.

7.2 Key Findings

Several findings stand out. Proving cost in EZKL scales cleanly with parameter count for MLPs but is driven, fundamentally, by circuit size, which in turn is governed by the structure of the computation rather than by numerical precision. Memory rather than compute is the binding constraint, which is why additional RAM extends the reachable model size while GPUs do not help. Attention and convolution are disproportionately expensive relative to their parameter counts, the former because of Softmax and the latter because convolutions unroll across spatial positions. Most importantly, neither reducing the fixed-point scale factor nor pre-quantizing weights reduces proving time, because Halo2 has no mechanism to exploit value-level redundancy; meaningful speedups require structural change to the model or a specialized proof system, exactly the path that zkLLM takes.

7.3 Limitations

Several limitations should be stated plainly. The benchmarking was bounded by the 18 GB of RAM on the test machine, which capped the MLP series at around two million parameters and prevented completion of the larger CNN models; some memory figures are therefore approximate and a few configurations are extrapolated rather than measured. The models used for benchmarking carried random weights, which is sound for measuring proving cost but means the study says nothing about model accuracy. The optimization experiments were conducted primarily on a single representative model, `m1p_w05`; while we expect the conclusions to generalize, confirming this across more architectures would strengthen them. Finally, our comparison with zkLLM is based on its published results rather than on our own re-execution, since reproducing zkLLM was out of scope.

7.4 Future Work

Several directions follow naturally from this work. The most direct is to lift the memory ceiling by running the benchmarks on a high-memory machine, which would extend the measured scaling curves from two million toward ten million parameters and complete the CNN and attention series. A second direction is to test structural rather than value-level optimization, in particular architectural pruning, which our analysis predicts should reduce proving time precisely because it changes circuit structure. A third is to explore alternative proof backends in the Nova and JOLT families, which promise better memory scaling than Halo2 and may shift the practical frontier. Finally, building the agent demonstration into a polished, live application would turn the system from a proof of concept into a compelling showcase of trustless AI, and would connect this work to emerging on-chain agent-accountability standards. Taken together, these directions point toward making verifiable inference progressively more practical, which is a necessary step if we are to trust the autonomous AI agents we increasingly rely on.

Bibliography

- [1] Ethereum Improvement Proposals. *ERC-8004: Trustless Agents*. <https://eips.ethereum.org/EIPS/eip-8004>. 2025.
- [2] Jens Groth. “On the Size of Pairing-Based Non-interactive Arguments”. In: *Advances in Cryptology – EUROCRYPT 2016*. Springer, 2016, pp. 305–326. DOI: 10.1007/978-3-662-49896-5_11.
- [3] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. *Scaling up Trustless DNN Inference with Zero-Knowledge Proofs*. 2022. arXiv: 2210.08674 [cs.LG]. URL: <https://arxiv.org/abs/2210.08674>.
- [4] Tianyi Liu, Xiang Xie, and Yupeng Zhang. “zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy”. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. Association for Computing Machinery, 2021, pp. 2968–2985. DOI: 10.1145/3460120.3485379.
- [5] ONNX Community. *ONNX: Open Neural Network Exchange*. <https://onnx.ai>. 2019.
- [6] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. <https://pytorch.org>. 2019.
- [7] Haochen Sun, Jason Li, and Hongyang Zhang. *zkLLM CUDA Implementation (CCS 2024)*. <https://github.com/jvhs0706/zkllm-ccs2024>. 2024.
- [8] Haochen Sun, Jason Li, and Hongyang Zhang. “zkLLM: Zero Knowledge Proofs for Large Language Models”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Also available as arXiv:2404.16109. Salt Lake City, UT, USA: Association for Computing Machinery, 2024, pp. 4405–4419. DOI: 10.1145/3658644.3670334.
- [9] Zcash Foundation and Electric Coin Company. *The Halo2 Book*. <https://zcash.github.io/halo2/>. 2022.
- [10] Zkonduit Inc. *EZKL: Easy Zero-Knowledge Inference*. <https://github.com/zkonduit/ezkl>. Documentation at <https://docs.ezkl.xyz>. 2024.